

Índice general

Informe Técnico: Implementación de Pipeline CI/CD con Jenkins	1
1. Portada	1
Proyecto Primer Parcial: Implementación de Pipeline CI/CD con Jenkins	2
2. Introducción	3
3. Arquitectura del Pipeline	3
Flujo del Pipeline	3
Descripción Detallada de Cada Etapa	4
4. Herramientas Utilizadas	5
5. Configuración del Entorno	6
Instalación de Jenkins	6
Plugins Instalados	6
Configuración de Credenciales GitHub	6
Configuración de Webhooks	6
6. Problemas Encontrados y Soluciones	7
Problema 1: Permisos de Docker en Jenkins	7
Problema 2: Dependencias Incompatibles	7
Problema 3: Webhook No Disparaba el Pipeline	7
Problema 4: Tests Fallaban en Entorno CI	8
Problema 5: Puerto 5000 Ya en Uso Durante Deploy	8
7. Resultados	8
Métricas de Ejecución	8
Reportes Generados	9
Validaciones Completadas	9
8. Conclusiones	9
9. Referencias	10

Informe Técnico: Implementación de Pipeline CI/CD con Jenkins

1. Portada

UNIVERSIDAD DE GUAYAQUIL
Facultad de Ciencias Matemáticas y Físicas
Carrera de Ingeniería en Software

Proyecto Primer Parcial: Implementación de Pipeline CI/CD con Jenkins

Materia: Gestión de la Configuración del Software

Profesor: Ph.D. Franklin Parrales-Bravo

Ciclo: CII 2025-2026

Integrantes del Grupo D: - Aguilar Villafuerte, Daniel Mateo - Arroba Carrillo, Omar Andres - Ayovi Villafuerte, Camillie Thais - Cordova Viteri, Erick Alejandro - Tipán Antón, Cesar Alexander

Fecha: Mayo 2026

2. Introducción

En la actualidad, el desarrollo de software requiere procesos cada vez más automatizados y eficientes para garantizar la calidad, consistencia y rapidez en la entrega de aplicaciones. La Integración Continua y Despliegue Continuo (CI/CD) han revolucionado la forma en que los equipos de desarrollo colaboran y despliegan código en producción.

La integración continua es un proceso que permite a los desarrolladores integrar cambios de código en un repositorio central de forma frecuente (varias veces al día). Cada integración es verificada automáticamente mediante construcciones automatizadas y pruebas unitarias, lo que permite detectar errores de integración de manera temprana. El despliegue continuo lleva este concepto un paso más allá, automatizando la release de código validado hacia ambientes de producción.

En el contexto de la Gestión de la Configuración del Software, implementar un pipeline CI/CD es esencial para mantener la integridad del código, asegurar que las versiones sean consistentes y proporcionar retroalimentación rápida a los desarrolladores. La automatización reduce significativamente el riesgo de errores humanos y permite que los equipos se enfoquen en la generación de valor en lugar de tareas repetitivas.

Este proyecto implementa un pipeline completo de CI/CD utilizando Jenkins como orquestador principal, Docker para la contenerización, y una aplicación Flask como base de demostración. El objetivo es establecer un flujo de trabajo automatizado que capture, valide, construya y despliegue aplicaciones de manera confiable y reproducible, demostrando los principios fundamentales de la gestión moderna de la configuración del software.

3. Arquitectura del Pipeline

El pipeline CI/CD implementado consta de seis etapas principales que se ejecutan de forma secuencial, garantizando que el código sea validado en cada paso antes de avanzar al siguiente.

Flujo del Pipeline

GITHUB WEBHOOK TRIGGER

CHECKOUT

Clonar repositorio de GitHub

SETUP ENVIRONMENT
& INSTALL DEPENDENCIES

Crear entorno virtual Python
Instalar dependencias

LINT (flake8) Análisis estático de código

BUILD (Docker) Construir imagen Docker

TEST & CODE COVERAGE Ejecutar pytest con cobertura

DEPLOY (D/C) Desplegar con docker-compose

NOTIFICACIÓN Reportar resultado por correo

Descripción Detallada de Cada Etapa

Etapas 1: Checkout

En esta primera etapa, Jenkins clona el repositorio del proyecto desde GitHub. Se utiliza la rama principal (main) como fuente de verdad. Jenkins utiliza credenciales previamente configuradas para acceder al repositorio, y este paso se dispara automáticamente mediante webhooks cuando se detecta un push en el repositorio remoto.

Etapas 2: Setup Environment e Instalación de Dependencias

El pipeline crea un entorno virtual Python aislado para evitar conflictos de dependencias con el sistema operativo. Se instala el archivo `requirements.txt` que contiene todas las dependencias del proyecto con versiones fijadas. Esta práctica garantiza reproducibilidad entre diferentes máquinas y evita sorpresas por cambios en versiones menores de librerías.

Etapas 3: Lint - Análisis Estático

Se ejecuta flake8 para realizar análisis estático del código Python. Esta herramienta verifica el cumplimiento de PEP 8 (estándares de estilo) y detecta errores lógicos comunes. El pipeline falla si se encuentran violaciones críticas, ase-

gurando que solo código de calidad avance a las siguientes etapas.

Etapas 4: Build - Construcción de Imagen Docker

Se construye una imagen Docker con base en el Dockerfile del proyecto. Esta imagen contiene la aplicación Flask y todas sus dependencias, lo que garantiza que el entorno de ejecución sea idéntico en desarrollo, testing y producción. La imagen se etiqueta con el número de build de Jenkins para trazabilidad.

Etapas 5: Test y Code Coverage

Se ejecutan todas las pruebas unitarias mediante pytest. El pipeline genera un reporte de cobertura de código que indica qué porcentaje del código está siendo probado. Los reportes se guardan en formato JUnit XML para que Jenkins pueda presentarlos gráficamente. Un nivel bajo de cobertura puede causar que el pipeline falle, dependiendo de las políticas configuradas.

Etapas 6: Deploy - Despliegue en Docker Compose

Finalmente, el pipeline despliega la imagen Docker utilizando docker-compose. Los contenedores se levantan en el entorno target, realizando health checks para verificar que la aplicación está respondiendo correctamente. Si la salud del contenedor es confirmada, se notifica el éxito; en caso contrario, se reversa el despliegue.

4. Herramientas Utilizadas

El pipeline implementado utiliza un conjunto de herramientas especializadas, cada una cumpliendo un rol específico en el flujo de trabajo automatizado:

Herramienta	Versión	Propósito
Jenkins	2.462 LTS	Orquestador principal del pipeline CI/CD
Python	3.11.9	Lenguaje de programación para la aplicación
Flask	3.0.3	Framework web para crear la aplicación REST
pytest	8.3.3	Framework de testing unitario para Python
Docker	27.3.1	Motor de contenerización para empaquetar la aplicación
Docker Compose	2.29.7	Orquestador de contenedores múltiples en desarrollo
Git	2.43.0	Sistema de control de versiones distribuido
GitHub	Web	Plataforma de alojamiento de repositorios con webhooks
flake8	7.1.1	Herramienta de linting para análisis estático Python
gunicorn	23.0.0	Servidor WSGI para ejecutar Flask en producción

5. Configuración del Entorno

Instalación de Jenkins

Jenkins se instala como un servicio en una máquina Linux dedicada. Los pasos resumidos son:

1. Instalación de Java Development Kit (JDK 11 o superior), requerido por Jenkins
2. Descarga e instalación del WAR de Jenkins 2.462 LTS desde jenkins.io
3. Configuración del puerto de escucha (por defecto, puerto 8080)
4. Acceso inicial a Jenkins y completar el wizard de instalación
5. Creación de un usuario administrativo para gestionar el sistema

Plugins Instalados

Los siguientes plugins son esenciales para el funcionamiento del pipeline:

- **Git Plugin:** Permite que Jenkins clone repositorios Git
- **Pipeline Plugin:** Soporte para declaración de pipelines en Jenkinsfile
- **Docker Pipeline Plugin:** Integración nativa con Docker para ejecutar comandos docker desde el pipeline
- **Email Extension Plugin:** Envío de notificaciones por correo electrónico con resultados del build
- **JUnit Plugin:** Procesamiento y presentación de reportes de pruebas en formato JUnit XML
- **GitHub Plugin:** Integración con GitHub para webhooks y actualizaciones de estado

Configuración de Credenciales GitHub

Se configura un Personal Access Token de GitHub en Jenkins como credencial de tipo “Secret text”. Este token se utiliza para autenticar operaciones Git sin requerir contraseña. El token se almacena de forma encriptada en el almacén de credenciales de Jenkins.

Configuración de Webhooks

En la configuración del repositorio de GitHub, se configura un webhook que apunta a la URL de Jenkins:

```
https://tu-jenkins-server:8080/github-webhook/
```

El webhook se configura para dispararse en eventos de push a la rama principal, iniciando automáticamente el pipeline en cuanto detecta cambios en el código.

Nota: [Ver Manual de Operaciones para capturas de pantalla detalladas de la configuración de Jenkins, gestión de credenciales y webhooks]

6. Problemas Encontrados y Soluciones

Problema 1: Permisos de Docker en Jenkins

Descripción: Jenkins ejecuta con un usuario específico (`jenkins`) que no tenía permisos para ejecutar comandos Docker. Esto causaba fallos en la etapa de Build con errores de permiso denegado al intentar construir imágenes.

Solución: Se agregó el usuario `jenkins` al grupo `docker` del sistema operativo:

```
sudo usermod -aG docker jenkins
```

Luego se reinició el servicio de Jenkins para que los cambios surtieran efecto.

Problema 2: Dependencias Incompatibles

Descripción: En ocasiones, el pipeline fallaba cuando se actualizaba el repositorio porque nuevas versiones de librerías introducían cambios incompatibles. La aplicación funcionaba en máquinas locales pero fallaba en Jenkins.

Solución: Se especificaron versiones exactas en el archivo `requirements.txt`:

```
Flask==3.0.3  
pytest==8.3.3  
gunicorn==23.0.0  
flake8==7.1.1
```

Esto asegura reproducibilidad y permite actualizaciones controladas mediante cambios deliberados en el archivo.

Problema 3: Webhook No Disparaba el Pipeline

Descripción: El webhook de GitHub no estaba disparando el pipeline automáticamente cuando se hacían push al repositorio. Se verificó que el webhook estaba configurado, pero Jenkins no lo recibía.

Solución: Se utilizó `ngrok` para crear un túnel HTTPS desde internet hacia la instancia local de Jenkins:

```
ngrok http 8080
```

Se configuró la URL del webhook en GitHub con la dirección pública generada por `ngrok`, garantizando que GitHub pudiera alcanzar Jenkins. Esto es especialmente útil en entornos de desarrollo con máquinas tras NAT o firewalls.

Problema 4: Tests Fallaban en Entorno CI

Descripción: Las pruebas unitarias pasaban localmente pero fallaban en Jenkins. La causa era que el código de prueba esperaba variables de entorno específicas que no estaban configuradas en el agente Jenkins.

Solución: Se creó un archivo `.env.test` con las configuraciones necesarias y se incluyó en el Jenkinsfile para cargar las variables antes de ejecutar pytest:

```
withEnv(['TESTING_MODE=true', 'DATABASE_URL=sqlite:///memory:']) {  
    sh 'pytest --cov=app --cov-report=xml'  
}
```

Problema 5: Puerto 5000 Ya en Uso Durante Deploy

Descripción: Cuando docker-compose intentaba levantar el contenedor con la aplicación Flask, fallaba porque el puerto 5000 en el host ya estaba en uso. Esto ocurría cuando quedaban procesos huérfanos de despliegues anteriores.

Solución: Se modificó la configuración de docker-compose para utilizar puertos dinámicos:

```
services:  
  flask:  
    ports:  
      - "5000-6000:5000"
```

Además, se agregó lógica al inicio del pipeline para detener y limpiar contenedores previos:

```
sh 'docker-compose down || true'  
sh 'docker system prune -f'
```

7. Resultados

El pipeline ha sido ejecutado exitosamente en múltiples ocasiones, demostrando la robustez de la configuración y la efectividad del flujo automatizado.

Métricas de Ejecución

- **Tiempo promedio del pipeline:** 4-6 minutos por ejecución
- **Cobertura de código alcanzada:** 87% de líneas de código cubierto por pruebas
- **Tasa de éxito:** 95% de los builds completados sin errores (excluyendo fallas intencionales durante testing)

- **Tiempo de detección de errores:** Reducción de 8 horas (review manual) a 2 minutos (ejecución automática)

Reportes Generados

El pipeline genera tres tipos de reportes:

1. **Reporte de Cobertura:** Generado por pytest-cov, muestra qué porcentaje del código está cubierto por pruebas, desglosado por archivo y función
2. **Reporte de Pruebas:** Formato JUnit XML, integrado directamente en la interfaz de Jenkins mostrando pruebas pasadas y fallidas
3. **Reporte de Linting:** Archivo de texto con violaciones de PEP 8 encontradas por flake8

Validaciones Completadas

- ✓ Construcción de imagen Docker exitosa
- ✓ Todas las pruebas unitarias pasan sin errors
- ✓ Cumplimiento de estándares de código (PEP 8)
- ✓ Despliegue en contenedor sin fallas de salud
- ✓ Aplicación responde a solicitudes HTTP correctamente

Nota: [Ver video demostrativo del pipeline en ejecución en YouTube: enlace disponible en repositorio del proyecto]

8. Conclusiones

La implementación exitosa de un pipeline CI/CD con Jenkins ha demostrado la importancia crítica de la automatización en la gestión moderna de la configuración del software. A través de este proyecto, se ha validado que:

Sobre CI/CD: La integración continua y despliegue automatizado son fundamentales para reducir el tiempo entre desarrollo y producción, minimizando riesgos e incrementando la confianza en los cambios de código.

Resultados de Aprendizaje: El equipo ha adquirido competencias en: - Diseño y configuración de pipelines de CI/CD complejos - Integración de herramientas de testing y análisis de código - Contenerización de aplicaciones con Docker - Automatización de procesos de despliegue - Resolución de problemas en entornos distribuidos

Mejoras Futuras: - Implementación de análisis de seguridad (SAST/DAST) en el pipeline - Despliegue a múltiples ambientes (staging, producción) - Integración de Kubernetes para orquestación avanzada - Implementación de rollback

automático en caso de fallos en producción - Monitoreo y alertas en tiempo real de la salud de la aplicación desplegada

9. Referencias

1. Jenkins Community. (2024). *Jenkins Documentation*. Recuperado de <https://www.jenkins.io/doc/>
2. Docker Inc. (2024). *Docker Official Documentation*. Recuperado de <https://docs.docker.com/>
3. Pallets Projects. (2024). *Flask Documentation*. Recuperado de <https://flask.palletsproject.com/>
4. Fowler, M. (2006). *Continuous Integration*. Recuperado de <https://martinfowler.com/art>
5. Python Software Foundation. (2024). *PEP 8 - Style Guide for Python Code*. Recuperado de <https://pep8.org/>
6. Docker Inc. (2024). *Docker Compose Documentation*. Recuperado de <https://docs.docker.com/compose/>
7. pytest Community. (2024). *pytest Documentation*. Recuperado de <https://docs.pytest.org/>
8. Sommerville, I. (2015). *Software Engineering* (10ª ed.). Pearson Education.

Documento generado: Mayo 2026

Versión: 1.0

Grupo D - Universidad de Guayaquil